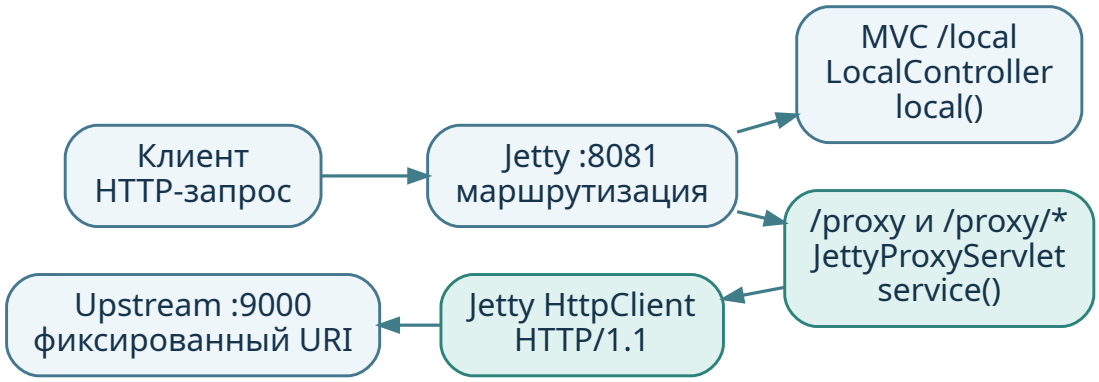
Servlet + Jetty HttpClient

Устройство и сопровождение модуля `jetty-client-example`

Практическое руководство по исходникам NettyPet. Как запускается приложение, кто вызывает каждый этап обмена, как передаются тела и как добавлять заголовки.

Версия: 24 октября 2026 · код модуля: `5d672f6` · Java 21 · Spring Boot 4.1.1 · Jetty 12.1.12.



Архитектура модуля

Это самостоятельный пример: встроенный Jetty принимает HTTP-запрос, Servlet управляет обменом, отдельный Jetty `HttpClient` обращается к фиксированному HTTP(S) upstream по HTTP/1.1. Тела передаются потоком. Spring Security пока не подключён.

Как читать

Страницы	Содержание
2–4	Карта исходников, запуск beans, путь одного запроса
5–7	Upload, download, потоки и владение памятью
8–9	Конвейер заголовков и пример расширения
10–12	Ошибки, отмена, shutdown и конфигурация
13–15	Запуск, проверка целостности, диагностика

Исходники и результаты проверок: PR #6. Редактируемая версия руководства и команда пересборки PDF находятся рядом с этим файлом в `docs/`.

Карта исходников: что искать

Все пути на этой странице отсчитываются от `jetty-client-example/`. Основной пакет — `src/main/java/dev/example/jettyclient/`; файлы прокси находятся в его подкаталоге `proxy/`.

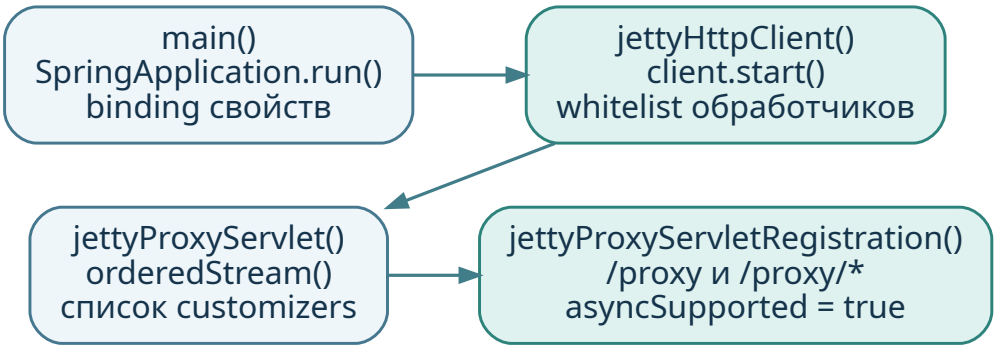
Файл / класс	Роль и основные точки входа
`JettyClientExampleApplication.java`	`main()` запускает Spring Boot. Вложенный `LocalController` обслуживает `/local`.
`proxy/JettyProxyProperties.java`	Binding и проверка `example.proxy`: адрес, размеры, лимиты и таймауты.
`proxy/JettyProxyConfiguration.java`	`jettyHttpClient()`, `jettyProxyServlet()`, `jettyProxyServletRegistration()` собирают beans.
`proxy/JettyProxyServlet.java`	`service()` принимает запрос; вложенный `Exchange` управляет upload, download, deadline и очисткой.
`proxy/Downstream.java`	`create()` выбирает native Jetty или переносимую Servlet-запись; `write()`, `flush()`, `cancel()`.
`proxy/ResponseContentBuffer.java`	`read()` и `demand()` читают ответ; ограниченное объединение уже доступных мелких chunks.
`proxy/ProxyHeaders.java`	`target()`, `request()`, `response()`: URI, фильтрация полей, вызов расширений.
`proxy/ProxyRequestMetadata.java`	Глубоко неизменяемый снимок method, target URI и исходных заголовков.
`proxy/ProxyHeaderCustomizer.java`	Hooks `customizeRequest()` и `customizeResponse()`.
`proxy/ProxyHeaderEditor.java`	Проверяемые `add()`, `set()`, `remove()`, `values()` для копии заголовков.
`proxy/InformationalProtocolHandler.java`	Обработка остальных промежуточных 1xx, кроме 101.

Настройки и доказательства поведения

- `src/main/resources/application.yml` — параметры и отключение MVC `FormContentFilter`.
- `src/test/java/dev/example/jettyclient/` — интеграционные проверки и header SPI; подкаталог `proxy/` — детерминированные проверки буферов.
- `README.md` — команды и контракт; `REVIEW.md` — итог ревью; `performance-results.json` — условия, SHA JAR и сырые замеры.

Модуль не зависит от `app`, `proxy` и `command-support`. В нём отсутствуют RequestProcessor, команды и Resilience4j из основного приложения. Перенос идеи в основной прокси потребует сохранить его отдельные контракты.

Запуск: от main до готового Servlet



Порядок создания компонентов

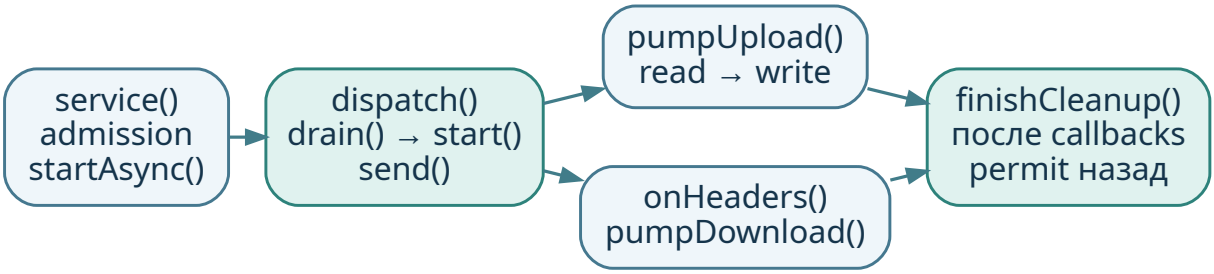
1. `JettyClientExampleApplication.main()` вызывает `SpringApplication.run()`. `@EnableConfigurationProperties` включает binding `JettyProxyProperties`.
2. `JettyProxyConfiguration.jettyHttpClient()` создаёт один общий клиент с `HttpClientTransportOverHTTP(selectors)`, TLS, лимитами и таймаутами.
3. Метод вызывает `client.start()`, затем очищает декодеры и protocol handlers. Это делается **после** start: Jetty устанавливает стандартные обработчики при запуске.
4. В whitelist возвращаются `ContinueProtocolHandler`, `ProcessingProtocolHandler`, `EarlyHintsProtocolHandler` и `InformationalProtocolHandler`. Они позволяют дождаться финального ответа после промежуточных `100/102/103` и остальных `1xx`.
5. `jettyProxyServlet()` получает `ObjectProvider<ProxyHeaderCustomizer>` и один раз собирает `orderedStream().toList()`. Servlet создаёт admission semaphore, реестр обменов и scheduler deadline.
6. `jettyProxyServletRegistration()` регистрирует `/proxy` и `/proxy/*`, `asyncSupported=true`, `loadOnStartup=1`.

Что настроено в клиенте

Redirect выключен, cookie store пустой, автоматическая распаковка и автоматическая аутентификация/upgrade отключены. Отсутствующие `User-Agent` и `Content-Type` не подставляются. Заголовки `Cookie` и повторные `Set-Cookie` текущего обмена передаются, но cookies не хранятся между пользователями.

HTTPS проверяет доверие сертификату и hostname: `SslContextFactory.Client(false)` и endpoint identification `HTTPS`. Режим trust-all не используется. При остановке контейнер вызывает `Servlet.destroy()`, а Spring останавливает клиент через `destroyMethod="stop"`; повторный Spring destroy Servlet отключён.

Один запрос: порядок вызовов



Основной путь обмена

Исходный Servlet-поток

`JettyProxyServlet.service()` проверяет CONNECT, Upgrade, Expect и framing, получает target через `ProxyHeaders.target()`, затем admission slot. Под `admissionGuard` остановка или заполнение лимита дают `503`. Создаётся `ProxyRequestMetadata` с исходными заголовками и фиксируется длина тела.

Далее `startAsync(request, response)` сохраняет **реальные** request/response, включая wrappers. Созданный `Exchange` попадает в реестр. `attachLifecycle()` синхронно устанавливает `AsyncListener` и `async.setTimeout(0)` до возврата из `service()`: общий deadline контролирует сам обмен.

Сериализованный владелец Exchange

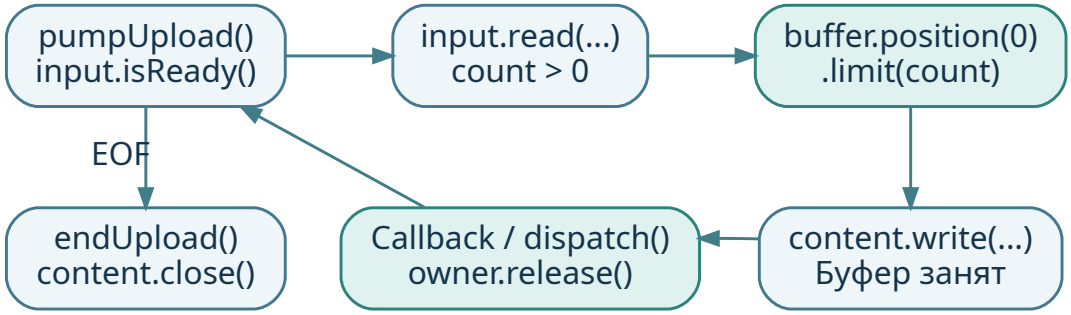
`dispatch(exchange::start)` ставит событие в очередь. В `start()` вычисляется оставшийся общий срок, планируется deadline, выбирается `Downstream` и вызывается `ProxyHeaders.request()` с request hooks. Затем создаётся upstream `Request`. Если тело известно по длине или framing, подключаются `StreamingContent` и `ReadListener`. `Request.send(this)` запускает обмен, `pumpUpload()` — чтение тела.

`onHeaders()` и `onContentSource()` запускают ответную половину; их события передаются тому же владельцу. Успех или `fail()` завершают async, а `finishCleanup()` отдельно ждёт возврата ресурсов перед освобождением admission.

Пример маршрутизации

При upstream `http://127.0.0.1:9000/base` запрос `/proxy/a?x=1&x=2` идёт в `/base/a?x=1&x=2`; `/proxy` и `/proxy/` — в `/base/`. `/proxying` не проксируется. Принятый контейнером raw path/query сохраняет `%XX` без двойного кодирования. Неоднозначный URI контейнер может отклонить ещё до Servlet.

Upload: браузер → upstream



Передача порции запроса

`ReadListener.onDataAvailable()` и `onAllDataRead()` вызывают `dispatch(pumpUpload)`. Насос проверяет состояние обмена, `input.isReady()` и отсутствие `uploadPending`. В полёте находится максимум одна upload-порция.

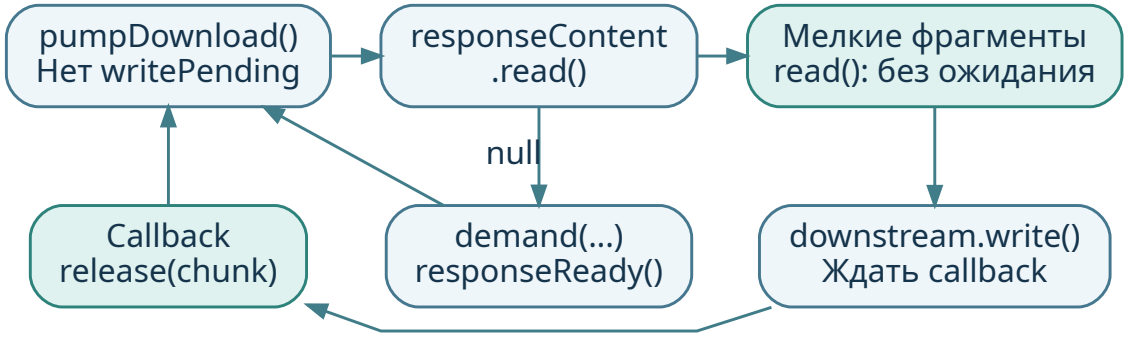
1. Из `ByteBufferPool` берётся буфер. Для точного класса Jetty `HttpInput` используется direct `ByteBuffer` и `read(ByteBuffer)`; для другого класса input — heap-массив и переносимый `read(byte[], …)`.
2. Размер чтения ограничен `chunkBytes`, даже если pool вернул большую ёмкость. После положительного чтения выставляются **`position(0).limit(count)`**. Здесь нельзя механически заменить подготовку буфера на `flip()`: поведение конкретного Jetty и стандартного Servlet API отличается.
3. `StreamingContent.write(buffer, callback)` отдаёт порцию клиенту upstream. Пока callback не сработал, буфер нельзя менять или возвращать pool.
4. Callback освобождает `owner` и снимает `uploadPending`. Успешный callback продолжает pump; ошибочный вызывает `fail(502)`, если обмен ещё активен и upload не остановлен ранним ответом. `once()` защищает от повторного завершения.
5. EOF приводит к `endUpload()`: проверке объявленной длины и `content.close()`. Избыток или недобор относительно Content-Length обрабатывается как `400`.

Почему большие тела не требуют большой сборки

`StreamingContent` наследует `AsyncRequestContent`, а `getLength()` сохраняет известную длину либо `-1` для неизвестной. Прокси не собирает тело полностью, не разбирает JSON/multipart и не меняет сжатые байты. Отсутствующий Content-Type остаётся отсутствующим. Границы HTTP chunked формируются заново на каждом соединении.

Это обратное давление: следующая порция upload не читается до подтверждения предыдущей. Подтверждение возвращает владение памятью; оно не доказывает, что удалённое приложение уже обработало эти байты.

Download: upstream → браузер



Чтение ответа и обратное давление

`onHeaders()` снимает snapshot финального статуса и полей. Владелец вызывает response hooks, затем устанавливает статус и заголовки downstream. `onContentSource()` передаёт `Content.Source` в `ResponseContentBuffer`. `pumpDownload()` работает по модели **read / demand**, без callback на каждый HTTP chunk.

- `read()` вернул `null`: регистрируется один demand; callback через `dispatch()` вызывает `responseReady()` и следующую попытку чтения.
- Получен chunk: `Downstream.write()` начинает запись. Его callback освобождает chunk, снимает `writePending` и продолжает pump. Дополнительный `retain()` не нужен: `Content.Source.read()` уже передал chunk вызывающему.
- Заголовки готовы, а тело задерживается: `Downstream.flush()` публикует заголовки без ожидания заполнения буфера. HEAD/204/205/304 обрабатываются как ответы без тела.

Объединение мелких фрагментов

Helper объединяет только уже доступные фрагменты **меньше 1024 байт**: не больше `chunkBytes` и максимум 1024 просмотренных фрагмента за проход. Одиночный доступный маленький chunk отправляется сразу. Новые данные ради заполнения batch не ожидаются. Крупные chunks передаются без копирования для batch; возможен один отложенный chunk (`carry`).

Два способа записи

Native путь выбирается только для исходного Jetty `ServletApiResponse`, пока output не выбран: `ServletCoreResponse.write(false, buffer, callback)`. Wrappers не обходятся. Portable путь использует настоящий `ServletOutputStream`, `WriteListener` и собственный непулируемый scratch-массив. Он копирует данные и соблюдает readiness. Поэтому весь маршрут нельзя объявлять zero-copy; callback означает возврат памяти, а не TCP delivery.

Потоки, владение и пределы памяти

`Exchange.dispatch()` принимает события из Servlet callbacks, HttpClient callbacks и scheduler. Очередь и счётчик работы запускают `drain()`, который выполняет действия одного обмена последовательно. События разных обменов могут идти параллельно. Это не отдельный поток на запрос и не привязка к одному физическому потоку.

Обычно `drain()` работает в executor `HttpClient`; при rejection используется commonPool для поздней очистки. Callback не запускает drain inline под сетевыми блокировками Jetty. `servletGuard` защищает работу с Servlet и переход в недоступное состояние. `containerFinished()` синхронно запрещает дальнейший Servlet I/O, после чего ставит завершение в очередь.

Ресурс	Кто владеет / когда освобождает
Admission permit	Exchange; только один `finishCleanup()` после всех условий
Upload pool buffer	Callback `AsyncRequestContent.write()`; до него повторное чтение запрещено
Chunk из `Content.Source.read()`	Сначала helper/Exchange, после передачи — callback downstream write
Aggregate мелких chunks	Одна pool reference; возвращённый chunk освобождается после write callback
Carry	Helper; отдаёт при следующем read или освобождает при отмене
Portable scratch	Downstream; не переиспользуется до readiness, после cancel отбрасывается
Header editor	Только текущий hook-поток; закрывается после конвейера

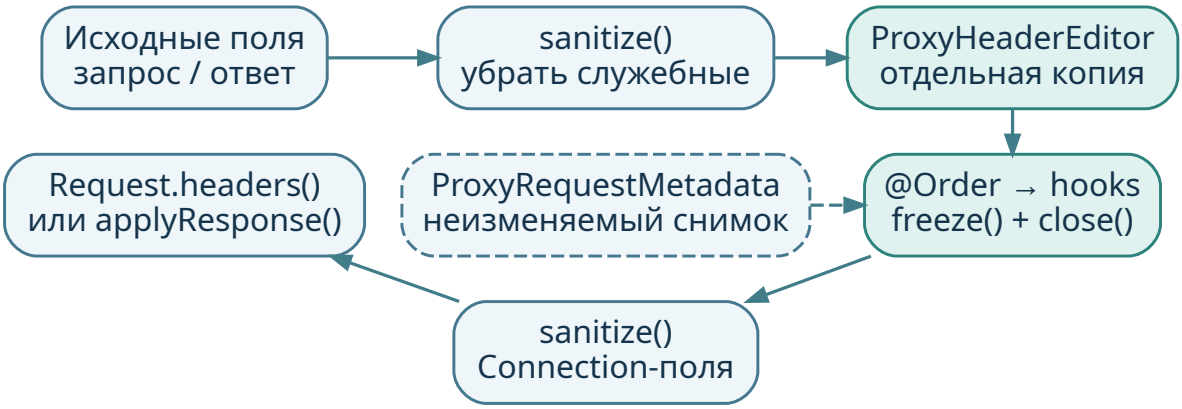
Практический смысл ограничений

Upload удерживает одну порцию. Download держит одну запись в полёте; batching ограничивает aggregate и lookahead. Это пределы **собственных полезных данных**, а не всего расхода памяти: parser, pool, TLS, Servlet wrappers, сокеты и JVM имеют свои буферы. Нельзя оценивать RSS формулой `maxConcurrent × chunkBytes`.

Header hooks обязаны быть быстрыми: блокировка владельца задерживает не только тело, но и обработку отмены/deadline. Вынести медленный код в новый поток и передать туда редактор тоже нельзя — он проверяет поток и закрытое состояние.

Особый случай — upstream `101`: `onHeaders()` немедленно вызывает `response.abort()` без Servlet и `servletGuard`, затем ставит локальный `502` в очередь. Отмена запрашивается прямо в callback без ожидания future; завершение и очистка выполняются асинхронно.

Заголовки: порядок и границы расширения



Конвейер изменения заголовков

`ProxyHeaders.request()` запускает request hooks до отправки upstream. `ProxyHeaders.response()` запускает response hooks для финального ответа **до установки статуса и полей downstream**. Без customizer beans редактор не создаётся.

В обоих направлениях порядок одинаков: исходные поля очищаются, создаётся отдельный `ProxyHeaderEditor`, hooks вызываются по возрастанию `@Order`, результат замораживается, повторно очищается и применяется. Редактор закрывается в `finally`. Каждый hook вызывается один раз на соответствующую фазу принятого обмена, если обработка дошла до этой фазы.

`ProxyRequestMetadata` содержит `method`, готовый upstream `target` и глубокий неизменяемый снимок исходных headers. Ключи `headers()` — в нижнем регистре; `headerValues(name)` ищет без учёта регистра. Response hook получает тот же исходный снимок, а не изменённые request hook поля.

Операция редактора	Поведение
`set(name, value)`	Заменить все значения этого имени
`add(name, value)`	Добавить значение, сохранив предыдущие
`remove(name)`	Удалить все значения без учёта регистра
`values(name)`	Получить неизменяемый снимок текущих значений

Что защищено

Нельзя менять `Host`, `Content-Length`, `Expect`, hop-by-hop поля и описание непрозрачного тела: `Content-Type`, `Content-Encoding`, `Content-Range`. Поля, перечисленные исходным `Connection`, удаляются и после hooks. Повторные `Set-Cookie` сохраняются отдельными значениями.

Неверное имя, CR/LF, управляющие или не представимые в ISO-8859-1 символы значения, защищённая мутация и исключение hook дают локальный `500` до commit. API не даёт менять URI, статус, тело или lifecycle. Beans общие для всех обменов: их состояние должно быть потокобезопасным; сеть, БД и чтение тела в hooks исключены.

Как добавить новый заголовок

В пакете `dev.example.jettyclient` создайте Spring-конфигурацию. Ниже полный пример: первый bean ставит service header, второй использует его значение. Порядок одинаков для запроса и ответа.

```
package dev.example.jettyclient;

import dev.example.jettyclient.proxy.ProxyHeaderCustomizer;
import dev.example.jettyclient.proxy.ProxyHeaderEditor;
import dev.example.jettyclient.proxy.ProxyRequestMetadata;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;

@Configuration(proxyBeanMethods = false)
class ExampleHeaders {
    @Bean @Order(10)
    ProxyHeaderCustomizer service() {
        return new ProxyHeaderCustomizer() {
            public void customizeRequest(ProxyRequestMetadata r,
                                         ProxyHeaderEditor h) {
                h.set("X-Example-Service", "gateway");
            }
            public void customizeResponse(ProxyRequestMetadata r,
                                          int status, ProxyHeaderEditor h) {
                h.add("X-Example-Service", "gateway");
            }
        };
    }

    @Bean @Order(20)
    ProxyHeaderCustomizer stage() {
        return new ProxyHeaderCustomizer() {
            public void customizeRequest(ProxyRequestMetadata r,
                                         ProxyHeaderEditor h) {
                if (h.values("X-Example-Service").contains("gateway"))
                    h.add("X-Example-Stage", "after-service");
            }
            public void customizeResponse(ProxyRequestMetadata r,
                                          int status, ProxyHeaderEditor h) {
                h.set("X-Example-Upstream-Status", "" + status);
            }
        };
    }
}
```

Для удаления поля используйте `h.remove("X-Legacy-Header")`. Редактор и ссылки на его изменяемое состояние нельзя сохранять после hook. Добавьте проверку порядка, повторных значений и ошибки расширения по образцу `JettyProxyHeaderCustomizationTest`; изменение тела требует отдельного дизайна.

Ошибки и ранний ответ upstream

Условие	Действие до commit
CONNECT, Upgrade / `Connection: upgrade`	`501` — неподдерживаемый режим
Любой входящий `Expect`	`417`; в curl upload используйте `-H 'Expect:'`
Неподдерживаемый request transfer encoding, ошибка длины/read upload	`400`
Заполнен admission, остановка, `RejectedExecutionException`	`503`
Общий deadline / `TimeoutException`	`504`
Ошибка header customizer	`500`; при request hook upstream ещё не отправлен
Иная транспортная ошибка, неподдерживаемый upstream `101`	Обычно `502`
Обычный HTTP-статус upstream, например 404 или 500	Передаётся как ответ upstream

`status(Throwable)` распознаёт конкретные типы исключений: не любой сетевой таймаут обязательно превращается в `504`. Ошибка `attachLifecycle()` даёт `500`; подготовка внутри `start()` использует общий `status()` и обычно даёт `502`. Промежуточные `1xx` ожидают финального ответа; downstream они не обещаются.

Пример: 413 до конца upload

Клиент ещё передаёт большое тело, а upstream уже ответил `413` с пояснением. `onHeaders()` прекращает **новые** upload reads, отмечает `uploadStopped` и ставит downstream `Connection: close`. Download продолжает доставлять тело ответа.

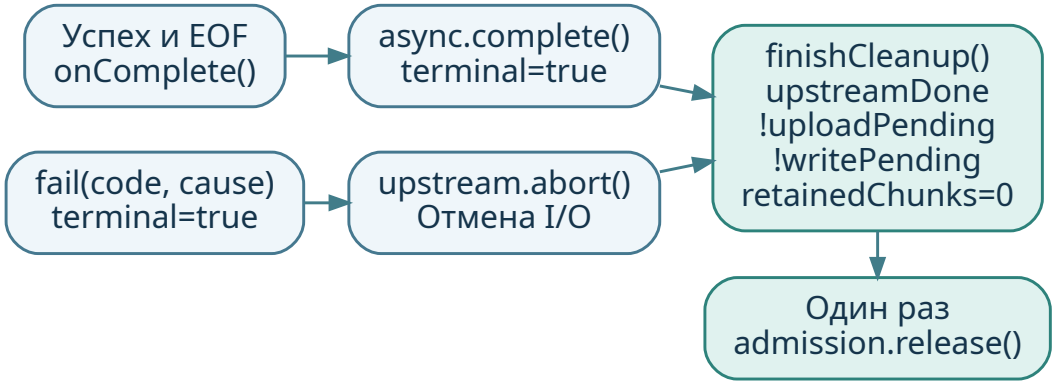
Только `onSuccess()` полностью полученного ответа вызывает `upstream.fail()` для незавершённого upload. Более ранняя остановка отправителя могла бы уничтожить и полезный ответ. `onComplete()` допускает намеренную request failure, если ответ успешно получен и response failure отсутствует.

Если часть ответа уже отправлена

`fail()` до commit сбрасывает response и отдаёт пустой локальный error response. После commit заменить статус уже нельзя: `abortDownstream()` обрывает канал через Jetty Servlet channel. Обычный `async.complete()` при обрезанном chunked-теле создал бы ложный успешный EOF. Поэтому клиент должен обнаружить незавершённое тело, а не получить видимость корректного ответа.

Тела не переигрываются автоматически. Добавление retry для POST/PUT требует отдельной политики повторов и способа воспроизвести уже прочитанный поток.

Завершение, отмена и shutdown



Логические условия завершения

Успешный downstream завершается, когда `responseSucceeded`, `upstreamDone`, получен последний chunk/EOF (`responseContentEnded`) и нет pending write. Последний флаг helper после опережающего чтения сам по себе недостаточен: важен последний chunk, реально обработанный владельцем.

При ошибке `fail()` единожды переводит Exchange в terminal, отменяет deadline, освобождает pending chunk/carry, вызывает `content.fail()`, стабильный `Request.abort()` и `Downstream.cancel()`. Async заканчивается либо error response, либо обрывом уже committed канала. Поздние callbacks ещё должны вернуть свои буферы.

Когда возвращается admission permit

`finishCleanup()` проверяет одновременно:

```
terminal && upstreamDone
         && !uploadPending && !writePending
         && retainedChunks == 0
         && released.compareAndSet(false, true)
```

Затем Exchange удаляется из реестра и освобождает permit. Установка terminal сама по себе не означает возврат ресурсов. CAS защищает от повторного release.

`ResponseContentBuffer.fail()` освобождает только собственные ссылки, в частности carry; он **не вызывает старый `Content.Source.fail()`**. Его receiver уже мог быть переиспользован другим обменом. Сетевая отмена выполняется через исходный `Request`.

Остановка приложения

`JettyProxyServlet.destroy()` запрещает admission под guard, ставит `fail(503)` активным Exchanges, останавливает timer и вызывает `super.destroy()`. Spring отдельно останавливает общий `HttpClient`. `activeRequests()` показывает незавершённые обмены модуля; его ноль и возврат из destroy не являются гарантией физического TCP/TLS close из основного `ProxyCall`.

application.yml: что настраивать

Файл: `src/main/resources/application.yml`. Префикс — **`example.proxy`**; настройки основного Netty-прокси с префиксом `proxy` здесь не применяются.

Ключ	Default	Назначение
`upstream`	`http://127.0.0.1:9000`	Фиксированный HTTP(S) адрес; `PROXY_UPSTREAM` переопределяет default
`chunk-bytes`	`65536`	Upload-порция, response buffer клиента и portable scratch; 1024…65536 байт
`max-concurrent`	`128`	Общий предел принятых незавершённых обменов, затем `503`
`max-connections-per-destination`	`64`	Предел upstream-соединений на destination
`max-queued-requests-per-destination`	`64`	Предел очереди ожидания соединения
`selectors`	`2`	Selectors отдельного клиента; не все потоки приложения
`connect-timeout`	`3s`	Таймаут установки соединения HttpClient
`exchange-timeout`	`60s`	Абсолютный срок обмена, включая очередь и тела
`idle-timeout`	`30s`	Отсутствие сетевого трафика, также при активном обмене

Destination limits общие, а не на selector. Это другая модель, чем Netty event-loop пулы основного приложения. Отдельного таймаута ожидания заголовков здесь нет. Счётчики положительные; длительности должны быть не меньше 1 мс и помещаться в миллисекунды `long`. Невалидная конфигурация отклоняется при binding/создании properties.

Upstream допускает base path, но не user info, query и fragment; нужны HTTP(S) и корректный host. Прокси не выбирает target из входящего Host или query.

Остальные настройки модуля

`server.address=127.0.0.1`, `server.port=8081`, лимит заголовков `16KB`, server compression выключена. `spring.mvc.formcontent.filter.enabled=false` отключает глобальный фильтр, который иначе мог прочитать form PUT/PATCH/DELETE до Servlet. Эта настройка влияет на всё standalone-приложение, включая его MVC-маршруты.

```
"$NETTYPET_GRADLE" :jetty-client-example:bootRun \
  --args='--server.port=8082 --example.proxy.max-concurrent=32'
```

Увеличение concurrency не устраняет медленный upstream. Сначала проверьте очередь destination, CPU, медленных клиентов и пределы генератора нагрузки.

Сборка, запуск и быстрые проверки

Все команды выполняются из корня репозитория. Нужны JDK 21 и Gradle 8.14.3. В checkout есть `gradlew`, но отсутствуют JAR/properties из `gradle/wrapper`; используйте установленный Gradle или подготовленную облачную установку.

```
# В подготовленной облачной среде; локально: NETTYPET_GRADLE=gradle
NETTYPET_GRADLE=/workspace/.setup/gradle
"$NETTYPET_GRADLE" :jetty-client-example:test \
  :jetty-client-example:bootJar

# Upstream должен быть запущен отдельно.
PROXY_UPSTREAM=http://127.0.0.1:9000 \
  java -jar jetty-client-example/build/libs/jetty-client-example-1.0.0.jar
```

В другом терминале проверьте локальный маршрут и проксирование. Для второго запроса upstream должен обслуживать `/a%20b`; его собственный 404 тоже передаётся.

```
curl -i 'http://127.0.0.1:8081/local'
curl --path-as-is -i \
  'http://127.0.0.1:8081/proxy/a%20b?x=%2F&x=2'

# Замените payload.bin существующим файлом, /upload — маршрутом upstream.
curl --http1.1 -H 'Expect:' --data-binary @payload.bin \
  'http://127.0.0.1:8081/proxy/upload'
curl --http1.1 -X PUT -H 'Expect:' -H 'Transfer-Encoding: chunked' \
  --data-binary @payload.bin 'http://127.0.0.1:8081/proxy/upload'
curl --http1.1 -H 'Expect:' -F 'file=@payload.bin' \
  -F 'description=sample' 'http://127.0.0.1:8081/proxy/upload'
```

Multipart с независимым эталоном

```
python3 stress/multipart/prepare.py \
  --output /tmp/nettypet-multipart --size-mib 8
# Терминал 1: приёмник работает в foreground.
python3 stress/multipart/server.py \
  --manifest /tmp/nettypet-multipart/manifest.json --port 9001
# Терминал 2: запустите JAR с PROXY_UPSTREAM=http://127.0.0.1:9001
# Терминал 3: проверка.
python3 stress/multipart/smoke.py --data-dir /tmp/nettypet-multipart \
  --base-url http://127.0.0.1:8081/proxy/multipart
```

Fixture создаётся один раз в новом каталоге. Приёмник сравнивает размер и SHA-256 всего wire-тела и каждой части с независимым manifest. Smoke проверяет и порчу частей, boundary, длины и Content-Type. Этот fixture требует Content-Length; chunked upload проверяется отдельно интеграционным тестом.

Проверки и контрольная производительность

JMeter отправляет `multipart.body` как всё тело, без второй multipart-обёртки. План проверяет каждый POST/PUT по локальному manifest; успешный exit JMeter недостаточен — обязательно проверьте assertions через `check_results.py`. Нужны JMeter 5.6.3 и JDK 21. Используйте новые JTL/log пути для каждого прогона.

```
# Локально: JMeter должен быть в PATH, JAVA_HOME указывать на JDK 21.
# В подготовленной облачной среде:
export JAVA_HOME=/workspace/.setup/tools/jdk-21.0.9+10
NETTYPET_JMETER=/workspace/.setup/tools/apache-jmeter-5.6.3/bin/jmeter
"$NETTYPET_JMETER" -n -t stress/multipart/multipart-post-put.jmx \
  -Jhost=127.0.0.1 -Jport=8081 -Jpath=/proxy/multipart \
  -JdataDir=/tmp/nettypet-multipart -Jthreads=2 -Jramp=0 -Jloops=2 \
  -l /tmp/multipart-smoke.jtl -j /tmp/multipart-smoke.log
python3 stress/multipart/check_results.py /tmp/multipart-smoke.jtl
```

Что уже проверено для кода 5d672f6

Полный build: **285 тестов**, без failures/errors/skips; из них 40 в модуле. Дополнительно JAR прошёл 16 HTTP smoke-проверок и 8 multipart JMeter-запросов с hash-проверкой тела и частей. Это сохранённые результаты проверки кода, а не автоматический повтор тестов при сборке PDF.

Класс теста	Число	Основные сценарии
`JettyProxyIntegrationTest`	20	Raw URI, 8 МиБ, multipart POST/PUT, form PUT, chunked, gzip, cookies, redirect, 1xx/101, early 413, timeout, reset, shutdown
`JettyProxyHeaderCustomizationTest`	14	Порядок hooks, snapshot, repeated fields, protected mutations, sanitization, ошибки
`ResponseContentBufferTest`	6	Границы batching, немедленная одиночная порция, владение и отмена

GET 8 МиБ: диагностика, а не предел системы

Метрика	Основное приложение / Netty	Пример / Jetty
Запросов/с	165,4	190,1
CPU, мс/запрос	7,86	5,59
p95 / p99, мс	205 / 275	156 / 194
Ошибок	0	0

Общий стенд: 4 CPU, JDK 21.0.9+10, heap 256 МиБ–1 ГиБ, HTTP/1.1, порции 64 КиБ, 20 клиентов, прогрев и два 15-секундных прогона на вариант A/B/B/A. В экспериментальной копии JMX удалён лимит 300 rps. Размер проверялся всегда, SHA-256 — до/после нагрузки. Netty-приложение содержит commands/Resilience4j, пример — нет; Security нет в обоих. Это не изолированное сравнение транспортов. Условия, SHA и результаты — в `performance-results.json`.

Диагностика и дальнейшие изменения

Наблюдение	Что проверить первым
`/local` работает, proxy даёт 502	Upstream напрямую, порт, base path, TLS trust/hostname
404	Итоговый upstream path; 404 самого upstream передаётся
417 / 501	Убрать Expect / проверить CONNECT и Upgrade
500 после добавления bean	Защищённые поля, допустимость имени/значения, исключения hook
503	Admission, destination queue, медленные клиенты, освобождение `activeRequests()`
504	Общий deadline, зависший upstream, очередь соединений
Пустой/изменённый upload	Фильтры, читающие тело, FormContentFilter, boundary, лишняя multipart-обёртка
Задержка событий	`curl -N`, буферизация клиента и промежуточных прокси; batching не ждёт заполнения
Низкий rps	Прогрев, SHA JAR, настройки, JMeter timer, CPU/сеть генератора и upstream

Для ответа 64 КиБ однобайтовыми chunks финальный короткий контроль с двумя клиентами дал 110,8 rps Netty и 130,4 rps Jetty (около 5 секунд). Исторический провал Jetty до batching — 4,46 rps за 3,139 секунды. Эти короткие прогоны объясняют необходимость batching, но не задают SLA. RSS и число потоков после фаз — снимки, не пределы памяти и не доказательство меньшего числа потоков; `selectors=2` не означает два потока JVM.

Перед изменением архитектуры

- Новый обычный заголовок: bean `ProxyHeaderCustomizer`, порядок и проверка повторных значений; тела и lifecycle не затрагивать.
- Новая body transformation: заново определить владение буферами, обратное давление, Content-Length/Encoding/Range и проверить large/tiny/slow/cancel сценарии.
- Новый фильтр или Spring Security: проверки до Servlet, поддержка async и wrappers, отсутствие преждевременного чтения тела; для сессий отдельно решить CSRF.
- Изменение Jetty: проверить upload ByteBuffer position/limit, native writer, промежуточные ответы, поздние callbacks и повторное использование соединений.

Границы текущего примера

Нет Security, retries, CONNECT-туннелей, WebSocket или HTTP/2 upstream. Передача trailers и промежуточных ответов downstream не заявлена. Multipart и сжатые тела непрозрачны; исходные HTTP chunked-границы не сохраняются. Добавление этих возможностей требует новых контрактов и проверок.

Исходники: модуль на GitHub. Для пересборки документа: `python3 jetty-client-example/docs/build_guide.py`.